Peter Olasupo (301909871_p.olasupo9871@miva.edu.ng)

ADVANCED DATABASE SYSTEMS

CONTINUOUS ASSESSMENT – PORTFOLIO SUMMARY

Institution / Course	Miva Open University | MIT 8103 – Advanced Database Systems
Project / Student	Design and Implementation of a Simulated MediLab Hospital Laboratory Database System | Peter Olasupo
GitHub Repository	https://github.com/akpet4christ/Peter_Olasupo_301909871_Advance-Database-Portfolio.git

INTRODUCTION AND SELECTED CASE STUDY

The current practical portfolio has been produced as a contribution to the continuous assessment of MIT 8103 Advanced Database Systems module. The case study used within the portfolio is focused on the MediLab Hospital Laboratory System, which is a hypothetical hospital laboratory information system used for all five portfolio tasks. The system is meant to facilitate laboratory personnel and scientists, healthcare providers, administrative staff, and database administrators.

Key datasets consist of patient demographic data, provider data, laboratory orders, ordered tests, tests catalog information, specimens, laboratory staff, and test results. Key operations include registering and retrieving patient data, creation of laboratory orders, association of tests with orders, specimen management, workflow management, results registration/retrieval. Patient data used are completely fabricated for educational purposes.

SUMMARY OF THE FIVE PORTFOLIO ACTIVITIES

Portfolio 1 – Database Design and Modeling

Design of the relational structure of the MediLab data has been done via identification of key entities and their attributes and representation of relations among them through the Entity-Relationship Diagram (ERD). The database model consists of PATIENTS, PROVIDERS, LAB_ORDERS, ORDER_TESTS, TEST_CATALOG, SPECIMENS, LAB_STAFF, and TEST_RESULTS. The relational schema has been documented and key fields and foreign keys have been identified and implemented in PostgreSQL 18 using pgAdmin 4. Normalization documentation has been prepared, alongside with SQL test statements that check data integrity rules and schema integrity.

Portfolio 2 – Query Processing and Optimization

SQL queries have been used to access and analyze data in MediLab, to test database operations, and to explore query execution process in PostgreSQL. Execution plans have been analyzed using EXPLAIN command with consideration of the way indexes and access paths affect performance. Practical results have been used to demonstrate differences between sequential access and index-based access, as well as to show why one should consider the size of the data when interpreting query plans.

Portfolio 3 – Transactions and Concurrency

Transaction control in PostgreSQL has been explored. Transaction execution, COMMIT and ROLLBACK actions, and consequences of different transaction isolation levels have been considered. Information about locks in database has been explored, and interactions among multiple operations when users access/modify shared laboratory data have been considered. Such exercises have demonstrated the crucial importance of transaction boundaries and isolation/locking mechanisms for maintenance of data consistency in the process of laboratory orders and results processing.

Portfolio 4 – NoSQL and Advanced Data Models

MongoDB Community Server and MongoDB Compass have been installed and used to design and manage document-based data in the framework of MediLab case study. Databases/collections have been created, patient/laboratory order documents have been inserted/queried, document updates/deletions have been performed, aggregation pipeline has been constructed to group and count order data, validation rules have been enforced, and an index has been created. It will be possible to conduct comparison between document-based model in MongoDB and relational model in PostgreSQL and evaluate applicability of each of them in certain contexts.

Portfolio 5 – Distributed and Cloud Database Exercise

A replica set of MongoDB has been configured and its PRIMARY node status has been checked. Four patient documents have been migrated from the source database medlab_nosql to the target database medlab_sync; then record counts have been verified, source record has been updated and target record has been manually reconciled. Data distribution has been simulated by putting patient records into three different collections and verifying their counts. The source update/target reconciliation has also been performed to understand concepts like stale data, consistency, availability, partition tolerance, and failover.

In the cloud part of the exercise a MongoDB Atlas cluster and medlab_cloud database with patients collection have been created. A simulated patient record has been inserted and queried successfully using MongoDB Compass and

Mongosh. Comparative evaluation of MongoDB Atlas, Amazon DynamoDB, Azure Cosmos DB, and Amazon RDS has been carried out, and public, private, hybrid, and multi-cloud deployment models have been considered. Implementation scope: the replica-set exercise used a single-node replica set; the sharding exercise has been a simulation using three collections instead of production sharded cluster; synchronization has been performed manually rather than automatically or in real-time. These limitations are acknowledged to make sure that production-grade distributed infrastructure is not presented here.

TECHNOLOGIES AND TOOLS

PostgreSQL 18 and pgAdmin 4 have been used for relational database design, SQL, transactions, and query analysis. MongoDB Community Server, MongoDB Compass, and Mongosh have been used for NoSQL, aggregation, validation, indexing, and local distributed-database demonstrations. MongoDB Atlas has been used for cloud database deployment and remote access. GitHub has been used to maintain the repository, technical documentation, and practical evidence.

INDIVIDUAL TECHNICAL REFLECTION

The project allowed me to combine theoretical knowledge about databases with a practical healthcare information scenario. A major decision in the design phase has been made to use normalized relational structure for records having explicit relations, while MongoDB documents have been used to allow flexible data representation. In the implementation phase issues with configuration and queries have appeared and needed verification of settings, revision of commands, refreshing database views and results validation. The exercises with distributed and cloud databases have shown the difference between illustration of a concept in a local environment and deployment of a resilient and multi-node production system. My principal takeaway is that database design should be appropriate to the characteristics of the data, workload, integrity requirements, scalability requirements, and operational environment.

AI USE DECLARATION

AI tool employed: ChatGPT (OpenAI). ChatGPT has been used as a learning and support tool to explain database concepts, clarify SQL and MongoDB commands, provide troubleshooting, help to interpret technical outputs, and help with structuring and editing portfolio documentation. The topics included relational modeling, normalization, SQL and query plans, transactions and concurrency, MongoDB CRUD and aggregation, replication, synchronization, sharding, consistency, availability, and cloud deployment.

I have verified the guidance in the context of database environments and outputs available during practical work. I have executed the database configuration and operations described in this report, have verified command results in PostgreSQL, MongoDB Compass, Mongosh, and MongoDB Atlas, and have repeated or corrected steps when the operation has not yielded the expected result. I have described explicitly the limitations of a single-node replica set, simulated sharding and manual synchronization, rather than presenting them as production deployments. I am responsible for the accuracy, understanding and submission of this work and can explain the activities and results.

GITHUB REPOSITORY AND SUPPORTING EVIDENCE

The GitHub repository includes the five portfolio folders, their README files, practical documentation and supporting materials such as schemas, SQL scripts, models, query results, screenshots, configuration details, and test evidence when applicable. The repository is supposed to be available to the assessor before LMS submission. The URL of the public repository should be provided in the designated field of the first page and accessed in a signed-out or private browser window before submission.

Repository URL:

https://github.com/akpet4christ/Peter_Olasupo_301909871_Advance-Database-Portfolio.git

CONCLUSION

The five portfolios trace the path from relational database design and SQL optimization to transaction management, NoSQL data modeling, distributed database concepts, and cloud deployment. The simulated MediLab Hospital Laboratory System has been used as a case study to apply and compare database approaches. The work has improved my practical understanding of database implementation, data integrity, performance, scalability, and cloud-hosted data management.

References

1. Miva Open University. MIT 8103 Advanced Database Systems: First Semester Continuous Assessment Brief, 2026/2027 academic session.

2. OpenAI. ChatGPT. Used as an AI learning and documentation support tool; details of use and student verification are stated in the AI Use Declaration.